

ERIC's Special Participation B - ChatGPT-5.1 Pro on EECS 182 HW5 Coding

Testing Extended Thinking on Coding Problems

My Prompt to ChatGPT-5.1 Pro

Prompt: "Generate comprehensive solutions for HW5 coding problems (Problems 5 and 6). Include all code implementations, outputs, explanations, and analysis. Show your work thoroughly."

Model Response:

- Entered extended thinking mode (approximately 30 minutes)
 - Generated complete coding solutions with implementations
 - Single-shot output with working code and detailed analysis
 - No iteration or debugging required
-

Experimental Setup

LLM Tested: ChatGPT-5.1 Pro (with extended thinking)

Topic: EECS 182 Homework 5, Problems 5 & 6 (Coding Problems)

Date: December 2025

Deconfliction: ChatGPT-5.1 Pro claimed in class spreadsheet

I tested ChatGPT-5.1 Pro's extended thinking capability on the coding portion of HW5. Unlike written derivations, coding problems require:

- Correct Python/PyTorch syntax
- Understanding of deep learning frameworks
- Debugging and testing code
- Interpreting experimental results
- Generating meaningful visualizations

The model spent approximately 30 minutes reasoning, then produced a complete solutions document with all code, outputs, and analysis.

Key Findings

One-Shot Success Rate: 100%

Both coding problems solved correctly on first attempt:

- Problem 5 (Understanding Dropout): ✓ All 9 parts correct
- Problem 6 (Batchnorm, Dropout, Convolutions): ✓ All 4 parts correct

Code Quality:

- All Python/PyTorch code runs without errors
- Proper use of NumPy, PyTorch, matplotlib
- Correct tensor operations and dimensions
- Proper GPU/CPU handling
- Clean, readable code with comments

Analysis Quality:

- Correct interpretation of all experimental results
- Deep conceptual explanations (not just code)
- Connects theory to implementation
- Explains unexpected results (e.g., spiky training curves)

Problem 5: Understanding Dropout

What Was Tested:

- Implementing dropout from scratch
- Least-squares vs gradient descent solutions
- Analyzing weight distributions with/without dropout
- Understanding batch size effects
- Interpreting real-world dropout experiments

Model Performance:

Code Implementation: Perfect

- Correctly implemented dropout layer with probability scaling
- Proper gradient descent training loops
- Accurate loss tracking and visualization
- All numerical outputs match expected results

Conceptual Understanding: Exceptional

- Explained why w_2 becomes 10× larger with dropout
- Analyzed spiky vs smooth training curves
- Connected dropout to regularization theory
- Demonstrated understanding of optimizer implicit biases

Key Insight Generated: Model correctly identified and explained the dramatic weight reversal:

- Without dropout: $w_1 \approx 1.09$, $w_2 \approx 0.11$ (w_1 is 10× larger)
- With dropout: $w_1 \approx 0.37$, $w_2 \approx 3.67$ (w_2 is 10× larger!)

The explanation was sophisticated: dropout forces w_2 to compensate for scenarios where w_1 is dropped, leading to balanced contributions ($10w_1 \approx w_2$).

Problem 6: Batchnorm, Dropout, and Convolutions

What Was Tested:

- Drawing computational graphs
- Deriving backpropagation formulas
- Implementing CNN architectures
- Hyperparameter tuning procedures
- Analyzing experimental results

Model Performance:

Computational Graph: Correct and Complete

- All edges properly identified ($X \rightarrow \mu$, $X \rightarrow \sigma^2$, $\mu \rightarrow \sigma^2$, etc.)
- Correct flow from inputs to outputs
- Proper intermediate node placement

Optional Derivation: Fully Correct

- Complete backpropagation through batch norm derived
- All three gradient terms identified (direct, mean-centering, variance)
- Proper chain rule application throughout
- Implementation code matches mathematical formulation

CNN Architecture: Professional Quality

python

```
class ConvNet(nn.Module):  
    # 4 conv layers, batch norm, dropout, proper dimensions  
    # Clean forward pass with ReLU activations  
    # Correct pooling and dropout placement
```

Hyperparameter Tuning: Systematic and Realistic

- Described 8-phase tuning procedure
- Proper methodology (one-at-a-time, validation-based)
- Reasonable hyperparameter ranges
- Ablation study showing component contributions

Quality Assessment

Code Correctness: 100%

- Zero syntax errors across ~500 lines of code
- All tensor dimensions correct
- Proper PyTorch idioms (`.to(device)`, `.eval()`, etc.)

- No runtime errors or exceptions

Framework Understanding: Expert-Level

- Correct use of `torch.optim`, `torch.nn`, `torch.nn.functional`
- Proper training/validation loops with gradient zeroing
- Correct dropout behavior (`.train()` vs `.eval()` modes)
- Understanding of batch norm running statistics

Analysis Depth: Graduate-Level

- Not just "what" but "why" for all results
- Connected dropout to ensemble learning, regularization, co-adaptation
- Explained train-val gap changes
- Discussed optimizer implicit biases (SGD vs Adam)

Pedagogical Quality:

- Code is readable with meaningful variable names
 - Explanations precede or follow code appropriately
 - Results interpreted with statistical reasoning
 - Connects implementation details to theory
-

Specific Achievements

Problem 5 Highlights:

1. Correct Implementation:

- Dropout layer with proper probability scaling
- Training loops with loss tracking
- Visualization with matplotlib

2. Insightful Analysis:

- Explained weight reversal phenomenon
- Analyzed gradient variance ($1/\sqrt{\text{batch_size}}$ relationship)

- Discussed optimizer-dependent solutions at dropout=0

3. **Advanced Understanding:**

- Connected dropout to L2 regularization via column norms
- Explained discontinuity in solution space
- Analyzed implicit biases of different optimizers

Problem 6 Highlights:

1. **Complete Graph:**

- All 9 edges in computational graph identified
- Clear visual representation provided

2. **Full Derivation:**

- Optional backprop problem solved completely
- Three gradient terms properly identified
- Clean NumPy implementation provided

3. **Practical Design:**

- Realistic CNN architecture
- Systematic hyperparameter tuning
- Proper ablation study methodology

Misconceptions/Hallucinations: NONE DETECTED

Verified Against Answer Key:

- All numerical answers exact matches
- All code implementations correct
- All conceptual explanations accurate
- Zero fabricated information

Framework Knowledge:

- All PyTorch API calls correct

- No deprecated functions used
- Proper tensor manipulation throughout
- Accurate understanding of dropout/batchnorm behavior

Mathematical Accuracy:

- All gradient derivations correct
 - Matrix dimensions consistent
 - Statistical reasoning sound
-

Comparison: Written vs Coding Problems

Similarities:

- Both achieved 100% accuracy
- Both required extended thinking (~30 minutes)
- Both produced professional-quality solutions
- Zero errors in either domain

Differences:

Aspect	Written Problems (A)	Coding Problems (B)
Error types	Math errors, sign mistakes	Syntax errors, runtime bugs
Verification	Symbolic derivation	Executable code
Complexity	Pure mathematics	Framework knowledge + logic
Output format	LaTeX equations	Python code
Testing	Manual checking	Run and verify

Key Observation: Coding is arguably HARDER than written math because:

- Must satisfy compiler/interpreter (unforgiving)
- Requires framework-specific knowledge (PyTorch API)

- Needs consistent tensor dimensions throughout
- Must handle GPU/CPU device placement
- Requires debugging skills

Yet ChatGPT-5.1 Pro achieved perfect accuracy on both.

Implications for EECS 182 Coding Assignments

For Traditional Coding Homework: This capability essentially "solves" coding assignments. Any well-specified programming task (implement dropout, write training loop, etc.) can be perfectly solved.

What Still Works:

- Open-ended design problems ("design your own architecture")
- Experimental interpretation ("explain these unexpected results")
- Debugging exercises ("here's broken code, fix it")
- Research-style exploration ("try different approaches and compare")

What's Fully Automated:

- "Implement batch normalization forward/backward"
 - "Write a training loop for dropout"
 - "Complete this CNN class"
 - Any coding task with correct answer
-

Conclusion

ChatGPT-5.1 Pro with extended thinking completely solves graduate-level deep learning coding problems with perfect accuracy. The 30-minute thinking time produces:

- Syntactically correct, runnable code
- Professional-quality implementations
- Deep conceptual explanations
- Proper use of ML frameworks

- Insightful analysis of results

Key Observation: The model demonstrates expert-level understanding of both coding (PyTorch APIs, tensor operations) and concepts (dropout regularization, gradient flow, optimizer biases). This isn't pattern matching - it's genuine understanding translated into working code.

Bottom Line: Every coding problem was one-shot correctly. Zero syntax errors, zero runtime bugs, zero conceptual mistakes. Combined with Option A results on written problems, this represents complete automation of traditional technical coursework.

Implications: Course assessment must shift away from "implement X" toward creative design, experimental interpretation, and open-ended research problems where there's no single correct answer.